

PHẦN 02

ĐỊNH DẠNG DỮ LIỆU XUẤT NÂNG CAO

| Chương 3 - Nhập xuất dữ liệu trong Python



TẠI SAO CẦN ĐỊNH DẠNG DỮ LIỆU?

DỮ LIỆU THÔ (RAW)

Pi: 3.141592653589793

Giá: 1500000.0

Tên: An Tuổi: 20

Khó đọc, thiếu chuyên nghiệp, không phù hợp cho báo cáo kỹ thuật.

ĐỊNH DẠNG (FORMATTED)

Pi: 3.142

Giá: 1,500,000 VNĐ

Tên: An | Tuổi: 20

Rõ ràng, đồng nhất, dễ dàng so sánh và phân tích kết quả.

Định dạng tốt giúp truyền tải thông tin hiệu quả hơn

ĐỊNH DẠNG BẰNG F-STRING PHƯƠNG PHÁP HIỆN ĐẠI

CÚ PHÁP CƠ BẢN

Sử dụng tiền tố **f** trước chuỗi và đặt biến bên trong cặp ngoặc nhọn **{ }**.

```
f"Văn bản {biến} văn bản"
```

VÍ DỤ MINH HỌA

```
name = "An"  
age = 20  
print(f"Chào {name}, bạn {age} tuổi.")
```

Kết quả: Chào An, bạn 20 tuổi.

TÍNH TOÁN TRỰC TIẾP TRONG F-STRING

NHÚNG BIỂU THỨC VÀO CHUỖI

Bên trong cặp ngoặc nhọn {}, bạn có thể viết bất kỳ biểu thức Python hợp lệ nào. Python sẽ tính toán giá trị đó trước khi đưa vào chuỗi.

```
a = 7
b = 3
print(f"{a} + {b} = {a + b}")
# Kết quả: 7 + 3 = 10

name = "python"
print(f"Học {name.upper()} thật thú vị!")
# Kết quả: Học PYTHON thật thú vị!
```

ĐỊNH DẠNG SỐ THỰC VỚI F-STRING

CÚ PHÁP LÀM TRÒN SỐ

Sử dụng cấu trúc `{biến:.nf}` để kiểm soát số chữ số sau dấu phẩy thập phân.

```
pi = 3.14159265  
print(f"Giá trị Pi làm tròn: {pi:.3f}")
```

Giá trị Pi làm tròn: 3.142

VÍ DỤ TRONG KỸ THUẬT

```
force = 125.6789  
print(f"Lực tác động: {force:.2f} N")
```

Lực tác động: 125.68 N

** Lưu ý: Python tự động làm tròn lên nếu chữ số tiếp theo lớn hơn hoặc bằng 5.*

CĂN LỀ VĂN BẢN VỚI F-STRING

CÁC KÝ TỰ CĂN LỀ

Sử dụng cú pháp {biến:độ_rộng} kết hợp với:

- < : Căn trái (Left alignment)
- > : Căn phải (Right alignment)
- ^ : Căn giữa (Center alignment)

```
txt = "Python"  
# Căn giữa trong 20 khoảng trống  
print(f"|{txt:^20}|")  
# Căn phải trong 20 khoảng trống  
print(f"|{txt:>20}|")
```

THÊM KÝ TỰ ĐỆM (PADDING) TRONG F-STRING

KỸ THUẬT PADDING

Ngoài việc căn lề bằng khoảng trắng, f-string cho phép lấp đầy các vị trí trống bằng bất kỳ ký tự nào bạn muốn.

```
# Đệm bằng dấu '=' và căn giữa
title = "BÁO CÁO"
print(f"{title:=^20}")
# Kết quả: =====BÁO CÁO=====

# Đệm bằng số '0' cho mã số
id = 42
print(f"ID: {id:0>5}")
# Kết quả: ID: 00042
```

SỬ DỤNG PHƯƠNG THỨC .FORMAT()

CÚ PHÁP TRUYỀN THỐNG

Trước khi f-string ra đời, **.format()** là cách tiêu chuẩn để định dạng chuỗi trong Python.

```
# Sử dụng vị trí mặc định
txt1 = "Chào {}, bạn {} tuổi".format("An", 20)

# Sử dụng chỉ số (index)
txt2 = "{1} là tên, {0} là tuổi".format(20, "An")

# Sử dụng tên biến (keyword)
txt3 = "Điểm: {s}".format(s=8.5)
```

Mặc dù f-string nhanh hơn, **.format()** vẫn rất hữu ích khi chuỗi định dạng và dữ liệu được xác định ở các thời điểm khác nhau.

ĐỊNH DẠNG KIỂU C VỚI TOÁN TỬ %

CÁCH ĐỊNH DẠNG CỔ ĐIỂN

Đây là phương pháp định dạng chuỗi lâu đời nhất trong Python, kế thừa từ ngôn ngữ C. Dù hiện nay ít được dùng hơn f-string, bạn vẫn sẽ gặp nó trong các mã nguồn cũ.

KÝ TỰ	KIỂU DỮ LIỆU TƯƠNG ỨNG
%s	Chuỗi ký tự (String)
%d	Số nguyên (Integer)
%f	Số thực (Floating point)

```
name = "An"  
score = 9.5  
print("Sinh viên %s đạt %d điểm" % (name, score))  
# Kết quả: Sinh viên An đạt 9 điểm
```

SO SÁNH CÁC PHƯƠNG PHÁP ĐỊNH DẠNG CHUỖI

TIÊU CHÍ	F-STRING	.FORMAT()	TOÁN TỬ %
Cú pháp	f"{var}"	"{}".format()	"%s" % var
Phiên bản	Python 3.6+	Python 2.7+, 3.0+	Tất cả phiên bản
Tốc độ	Nhanh nhất	Trung bình	Chậm nhất
Độ dễ đọc	Rất cao	Cao	Thấp
Khuyến nghị	Nên dùng nhất	Dùng khi cần thiết	Hạn chế dùng

* Kết luận: Hãy ưu tiên sử dụng **f-string** cho các dự án hiện đại để tối ưu hiệu năng và độ sạch của mã nguồn.

ỨNG DỤNG: IN BẢNG KẾT QUẢ KỸ THUẬT

MÃ NGUỒN PYTHON

```
# Định nghĩa tiêu đề và dữ liệu
header = f"{'Vật liệu':<15} {'Độ bền (MPa)':>15} {'Hệ số an toàn':>15}"
line = "-" * 50
mat1 = f"{'Thép A36':<15} {250.00:>15.2f} {1.5:>15.1f}"
mat2 = f"{'Nhôm 6061':<15} {276.55:>15.2f} {2.0:>15.1f}"

print(header); print(line); print(mat1); print(mat2)
```

KẾT QUẢ HIỂN THỊ

```
Vật liệu Độ bền (MPa) Hệ số an toàn
```

```
Thép A36 250.00 1.5
```

```
Nhôm 6061 276.55 2.0
```

TÓM TẮT NỘI DUNG

CHƯƠNG 3 - PHẦN 2

SỨC MẠNH CỦA F-STRING

Phương pháp định dạng hiện đại nhất từ Python 3.6+. Cho phép nhúng biến và biểu thức trực tiếp vào chuỗi một cách trực quan và hiệu quả.

ĐỘ CHÍNH XÁC KỸ THUẬT

Sử dụng định dạng `:.nf` để kiểm soát số chữ số thập phân, cực kỳ quan trọng trong việc trình bày các kết quả tính toán khoa học.

TRÌNH BÀY CHUYÊN NGHIỆP

Kỹ thuật căn lề (`<`, `>`, `^`) và thêm ký tự đệm (**padding**) giúp tạo ra các bảng dữ liệu và báo cáo kỹ thuật rõ ràng, đẹp mắt.

TÍNH TƯƠNG THÍCH

Nắm vững phương thức `.format()` và toán tử `%` để làm việc với các dự án cũ hoặc các tình huống định dạng chuỗi động phức tạp.